LAPORAN PRAKTIK
PEMROGRAMAN BERORIENTASI OBJEK
SISTEM MANAJEMEN E-COMMERCE



Disusun Oleh:

KELOMPOK 1

ABDUL MAJID	**(225443001)**
FARHAN MAULANA	**(225443008)**
KIYOSHI OLOAN RAPHAEL SITORUS	**(225443012)**
MUHAMAD FARREL ATTALA	**(225443013)**
MUHAMMAD IHSAN JULIANSAH	**(225443017)**
MUHAMMAD IZZUDIN IRSYAD	**(225443018)**
NAUFAL RABBANI	**(225443022)**

DOSEN PENGAMPU:

MOHAMMAD HARRY KHOMAS SAPUTRA, S.T., M.TI

2AEC-1

TEKNOLOGI REKAYASA INFORMATIKA INDUSTRI

TEKNIK OTOMASI MANUFAKTUR DAN MEKATRONIKA

POLITEKNIK MANUFAKTUR BANDUNG

TAHUN AJARAN 2026/2027

DAFTAR ISI

DAFTAR ISI ii

BAB I PENDAHULUAN 1

1.1. Latar Belakang 1

1.2. Tujuan Praktikum 1

1.3. Ruang Lingkup Sistem 2

BAB II DESAIN SISTEM 3

2.1. Diagram Kelas (Class Diagram) 3

2.2. Deskripsi Kelas dan Hubungan Antar-Kelas 3

BAB III IMPLEMENTASI SISTEM 6

3.1. Modul Produk (Product Hierarchy) 6

3.2. Modul Keranjang Belanja (Shopping Cart) 6

3.3. Modul Pesanan dan Status (Order & OrderStatus) 7

3.4. Modul Gerbang Pembayaran (Payment Gateway) 7

3.5. Modul Pengguna dan Pengendali Sistem (User & ECommerceSystem) 7

3.6. Program Utama (Main.kt) 8

BAB IV DEMONSTRASI DAN PENGUJIAN 9

4.1. Tampilan Output Terminal 9

4.2. Analisis Skenario Pengujian Sistem 14

BAB V ANALISIS PENERAPAN PILAR OOP 16

5.1. Enkapsulasi (Encapsulation) 16

5.2. Pewarisan (Inheritance) 17

5.3. Polimorfisme (Polymorphism) 18

5.4. Abstraksi (Abstraction) 19

5.5. Fitur Lanjutan Kotlin: Sealed Class & Smart Casting 19

BAB VI KESIMPULAN DAN SARAN 21

6.1. Kesimpulan 21

6.2. Saran **21**

BAB VII LAMPIRAN **22**

7.1. File `src/product/Product.kt` **22**

7.2. File `src/product/ElectronicProduct.kt` **24**

7.3. File `src/product/ClothingProduct.kt` **25**

7.4. File `src/product/FoodProduct.kt` **27**

7.5. File `src/cart/ShoppingCart.kt` **28**

7.6. File `src/order/OrderStatus.kt` **31**

7.7. File `src/order/Order.kt` **33**

7.8. File `src/payment/PaymentMethod.kt` **35**

7.9. File `src/payment/CreditCardPayment.kt` **36**

7.10. File `src/payment/QRISPayment.kt` **37**

7.11. File `src/payment/BankTransferPayment.kt` **38**

7.12. File `src/user/User.kt` **39**

7.13. File `src/system/ECommerceSystem.kt` **42**

7.14. File `src/Main.kt` **46**

BAB I
PENDAHULUAN

1.1. Latar Belakang

Perkembangan teknologi perangkat lunak saat ini menuntut pembuatan aplikasi yang modular, mudah dikelola (*maintainable*), dan dapat diperluas (scalable). Paradigma Pemrograman Berorientasi Objek (*Object-Oriented Programming* / OOP) merupakan standar industri yang mengorganisasikan perangkat lunak ke dalam objek-objek diskrit yang merepresentasikan entitas nyata maupun abstrak beserta data (*state*) dan perilakunya (*behavior*).

Dalam domain aplikasi perdagangan elektronik (*e-commerce*), kompleksitas sistem melibatkan transaksi dinamis, keragaman jenis produk (elektronik, pakaian, pangan), variasi formula potongan harga, mekanisme pergerakan inventaris stok barang, keanekaragaman gerbang pembayaran, hingga pelacakan tahapan status pengiriman pesanan. Bahasa pemrograman Kotlin menawarkan dukungan modern yang kuat terhadap paradigma OOP murni, dikombinasikan dengan fitur keamanan tipe data tingkat lanjut seperti *Null-Safety*, *Sealed Classes*, dan *Smart Casting*.

Melalui tugas praktikum kelompok ini, dirancang dan dibangun sebuah purwarupa sistem manajemen e-commerce terstruktur yang mengimplementasikan secara utuh empat pilar utama OOP: Enkapsulasi, Pewarisan, Polimorfisme, dan Abstraksi, serta menerapkan standarisasi dokumentasi kode menggunakan KDoc.

1.2. Tujuan Praktikum

Tujuan pelaksanaan praktikum tugas kelompok ini adalah sebagai berikut:

1) Menerapkan prinsip Enkapsulasi untuk menyembunyikan data sensitif (seperti kata sandi akun pengguna dan basis harga produk) serta membatasi mutasi langsung melalui *access modifier* (`private`, `protected`) dan *private setter.*
2) Menerapkan prinsip Pewarisan (*Inheritance*) untuk membangun hierarki kelas yang efisien, mengurangi redundansi kode, serta memperluas kapabilitas kelas induk menggunakan kata kunci `open`, `abstract`, `override`, dan `super`.

3) Menerapkan prinsip Polimorfisme (*Polymorphism*) melalui referensi polimorfik (*polymorphic references*) dan penanganan metode dinamis (*dynamic dispatch*) untuk memproses objek produk dan gerbang pembayaran secara seragam.
4) Menerapkan prinsip Abstraksi (*Abstraction*) melalui *Abstract Class* dan *Interface* guna mendefinisikan kontrak perilaku yang wajib dipenuhi oleh subclass terkait.
5) Memanfaatkan fitur khusus bahasa Kotlin, yakni Sealed Class untuk memodelkan siklus status pesanan secara terbatas dan aman (*exhaustive when*), serta Smart Casting (`is`) dan Safe Casting (`as?`).

1.3. Ruang Lingkup Sistem

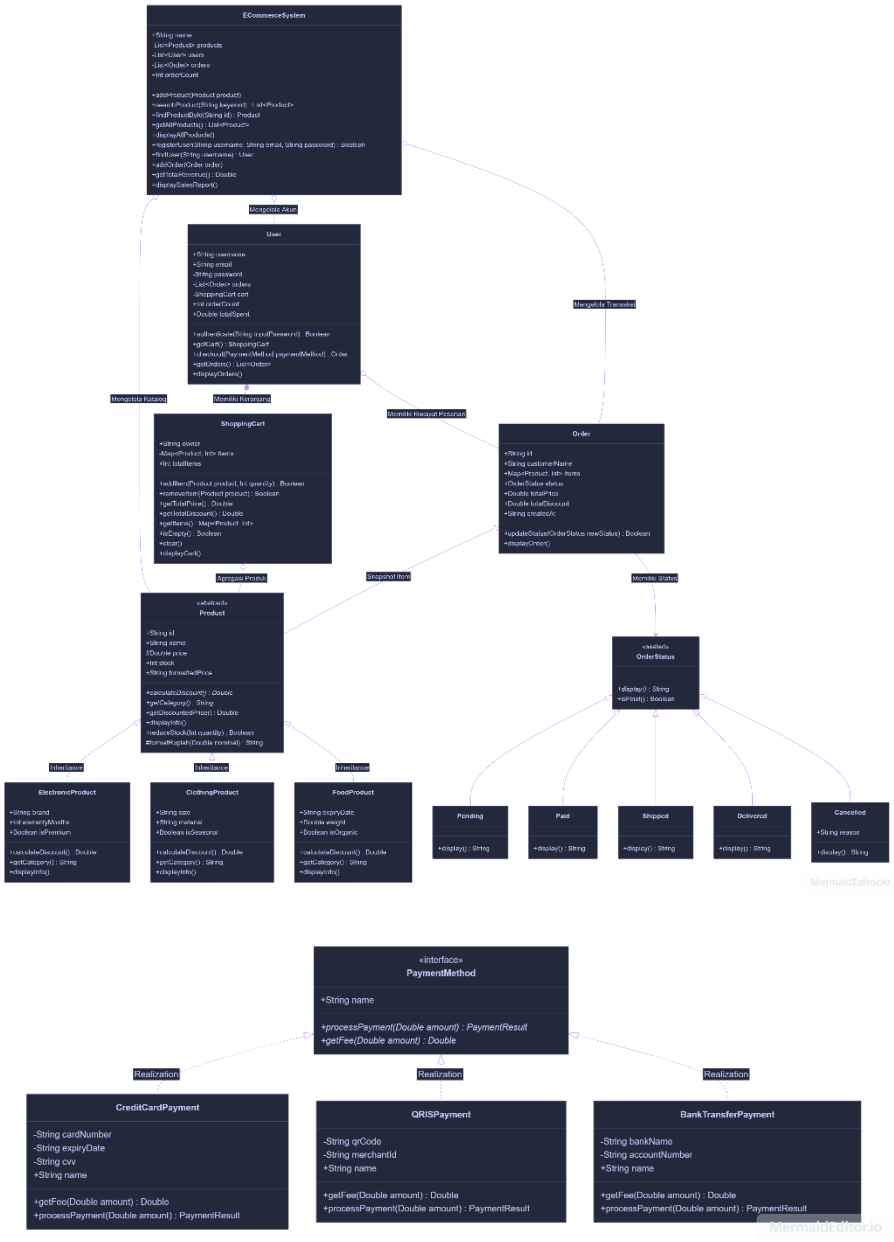
Ruang lingkup dari sistem manajemen e-commerce yang dikembangkan mencakup:

1) **Manajemen Produk:** Pengelolaan produk berkategori Elektronik, Pakaian, dan Makanan dengan atribut spesifik serta logika kalkulasi diskon yang berbeda untuk tiap kategori.
2) **Pengelolaan Keranjang Belanja:** Penambahan produk, pengurangan kuantitas, pengembalian stok barang secara otomatis saat pembatalan, dan perhitungan akumulasi biaya sebelum dan sesudah diskon.
3) **Pemesanan dan Siklus Transaksi:** Pembuatan pesanan unik (`ORD-...`), pencatatan waktu transaksi (*timestamp*), serta validasi siklus status pesanan (`Pending`, `Paid`, `Shipped`, `Delivered`, `Cancelled`) yang kebal terhadap manipulasi ketika sudah berstatus akhir (*final*).
4) **Gerbang Multi-Metode Pembayaran:** Integrasi kontrak interface pembayaran untuk transaksi Kartu Kredit (biaya layanan 2%), QRIS (biaya layanan 0.5%), dan Transfer Bank (biaya layanan 1% dengan batas minimal Rp 5.000).
5) **Autentikasi Akun Pengguna:** Perlindungan data kredensial pengguna, pencatatan histori pesanan personal, serta perhitungan total pengeluaran akumulatif.
6) **Laporan Penjualan Toko:** Rekapitulasi jumlah order dan total perolehan omset (*revenue*) toko online.

BAB II
DESAIN SISTEM

2.1. Diagram Kelas (Class Diagram)

Berikut adalah diagram kelas (*Class Diagram*) sistem manajemen e-commerce yang memvisualisasikan relasi pewarisan (*inheritance*), implementasi (*realization*), asosiasi (*association*), dan komposisi (*composition*):



2.2. Deskripsi Kelas dan Hubungan Antar-Kelas

Nama Kelas / Interface	Jenis	Deskripsi

`Product`	Abstract Class	Kelas induk produk; mengenkapsulasi harga (`protected price`), mendefinisikan metode abstrak `calculateDiscount()` dan `getCategory()`, serta mengelola pengurangan stok.
`ElectronicProduct`	Class (Subclass)	Mewarisi `Product`; mengelola produk elektronik dengan atribut garansi, merek, dan status premium, serta formula diskon berbasis masa garansi dan premium.
`ClothingProduct`	Class (Subclass)	Mewarisi `Product`; mengelola pakaian dengan atribut ukuran, bahan, dan status musiman (seasonal), serta formula diskon ukuran besar (XL/XXL/XXXL) dan musim.
`FoodProduct`	Class (Subclass)	Mewarisi Product; mengelola makanan dengan atribut kadaluarsa, berat bersih, dan status organik, serta formula diskon produk organik.
`ShoppingCart`	Class	Mengelola item belanjaan seorang pelanggan, memvalidasi kuantitas belanja, memotong stok secara real-time, dan mengembalikan stok jika item dibatalkan.
`OrderStatus`	Sealed Class	Memodelkan siklus status pesanan tertutup (`Pending`, `Paid`, `Shipped`, `Delivered`, `Cancelled`) dengan proteksi status akhir `isFinal()`.
`Order`	Class	Merepresentasikan nota transaksi pesanan yang sah, berisi snapshot daftar produk yang dibeli,

		penghitungan total harga, dan pelacakan status.
`PaymentMethod`	Interface	Kontrak abstraksi sistem pembayaran; mewajibkan metode `processPayment(amount)` dan `getFee(amount)`.
`PaymentResult`	Sealed Class	Hasil status pemrosesan transaksi pembayaran (`Success`, `Failed`, `Pending`).
`CreditCardPayment`	Class (Implementasi)	Implementasi pembayaran kartu kredit dengan validasi 16 digit dan CVV 3 digit (biaya layanan 2%).
`QRISPayment`	Class (Implementasi)	Implementasi pembayaran QRIS dengan validasi panjang kode QR minimal 10 karakter (biaya layanan 0.5%).
`BankTransferPayment`	Class (Implementasi)	Implementasi transfer bank dengan validasi nomor rekening minimal 8 digit (biaya layanan 1%, min. Rp 5.000).
`User`	Class	Entitas pelanggan; mengenkapsulasi kata sandi secara privat (`private password`), memfasilitasi autentikasi, serta menjalankan alur checkout.
`ECommerceSystem`	Class (Controller)	Pengendali utama sistem e-commerce; mengelola koleksi produk, pendaftaran user, pencarian produk, dan pembuatan laporan rekapitulasi penjualan toko.

BAB III
IMPLEMENTASI SISTEM

3.1. Modul Produk (Product Hierarchy)

Modul produk diorganisasikan di bawah paket `product`. Kelas induk abstrak `Product` mendefinisikan kerangka umum produk:

1) **Proteksi Harga:** Properti `price` dideklarasikan dengan modifier `protected` sehingga tidak dapat diakses langsung oleh kelas luar (seperti `Main`, `ShoppingCart`, atau `User`), namun tetap dapat diakses oleh subclass turunannya untuk keperluan kalkulasi diskon.
2) **Getter Terformat:** Properti `formattedPrice` mengembalikan teks harga dalam format mata uang Rupiah standar (contoh: "`Rp 15.000.000`").
3) **Aturan Bisnis Diskon Subclass:**
    - `ElectronicProduct`: Diskon dasar 5% + diskon premium 10% + bonus garansi panjang (> 24 bulan) 5%.
    - `ClothingProduct`: Diskon dasar 10% + diskon pakaian musiman 15% + diskon ukuran besar (XL, XXL, XXXL) 5%.
    - `FoodProduct`: Diskon dasar 5% + diskon sertifikasi organik 20%.

3.2. Modul Keranjang Belanja (Shopping Cart)

Modul keranjang belanja (`cart.ShoppingCart`) mengelola barang-barang yang dipilih pelanggan:

1) **Enkapsulasi Koleksi:** Properti items bertipe `private val items = mutableMapOf<Product, Int>()` sehingga pihak luar tidak dapat memanipulasi pasangan produk dan kuantitas tanpa melalui validasi.
2) **Private Setter pada Total Barang:** Properti `totalItems` memiliki deklarasi `var totalItems: Int = 0 private set`, memastikan jumlah total barang hanya diperbarui oleh internal metode `addItem()` atau `removeItem()`.
3) **Integritas Inventaris Stok:** Metode `addItem()` memanggil `product.reduceStock(quantity)`. Jika stok produk di gudang tidak mencukupi, penambahan barang akan ditolak. Sebaliknya, metode `removeItem()` akan mengembalikan stok produk yang dibatalkan.

3.3. Modul Pesanan dan Status (Order & OrderStatus)

Modul pesanan mengintegrasikan entitas nota transaksi dengan mesin status:

1) **Sealed Class `OrderStatus`**: Terdiri atas objek `Pending`, `Paid`, `Shipped`, `Delivered`, dan data class `Cancelled(reason)`. Sifat tertutup dari sealed class ini memungkinkan evaluasi percabangan `when` secara ekshaustif (*exhaustive evaluation*) pada saat kompilasi tanpa perlu cabang default `else`.
2) **Proteksi Status Final**: Metode `isFinal()` mengembalikan nilai `true` jika status berada pada kondisi `Delivered` atau `Cancelled`. Dalam kelas `Order`, fungsi `updateStatus()` akan memblokir perubahan status apabila status pesanan saat ini sudah final.

3.4. Modul Gerbang Pembayaran (Payment Gateway)

Modul pembayaran (`payment`) mengimplementasikan prinsip abstraksi murni:

1) Interface `PaymentMethod` mendefinisikan metode `processPayment(amount: Double): PaymentResult` dan `getFee(amount: Double): Double`.
2) Tiga gerbang pembayaran mengimplementasikan interface ini:
    - `CreditCardPayment`: Memvalidasi nomor kartu (minimal 16 digit) dan CVV (tepat 3 digit), mengenakan biaya layanan 2%.
    - `QRISPayment`: Memvalidasi panjang string payload kode QR (minimal 10 digit), mengenakan biaya layanan 0.5%.
    - `BankTransferPayment`: Memvalidasi nomor rekening (minimal 8 digit), mengenakan biaya layanan 1% dengan nominal batas minimum Rp 5.000.
3) Hasil pembayaran dienkapsulasi menggunakan sealed class `PaymentResult` yang membedakan status `Success`, `Failed`, dan `Pending`.

3.5. Modul Pengguna dan Pengendali Sistem (User & ECommerceSystem)

1) **Kelas `User`:** Menyimpan username, email, dan `password` bertipe `private`. Kata sandi tidak dapat dilihat dari luar kelas melainkan hanya dapat diuji kecocokannya melalui fungsi `authenticate(inputPassword)`. Fungsi `checkout(paymentMethod)` menghubungkan keranjang belanja dengan gerbang pembayaran, membentuk objek `Order`, dan mengosongkan keranjang belanja.

2) **Kelas `ECommerceSystem`:** Berperan sebagai fasad pengendali (*controller*) yang menampung katalog seluruh produk, akun pengguna terdaftar, serta riwayat seluruh transaksi order toko. Menyediakan fungsi pencarian produk fleksibel berbasis kata kunci (*case-insensitive*) dan rekapitulasi total pendapatan (*revenue*).

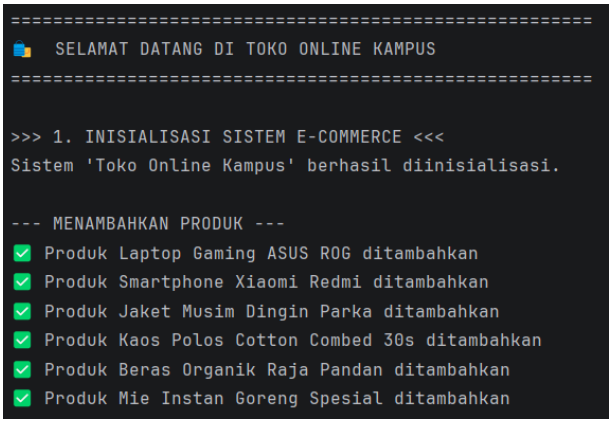
3.6. Program Utama (Main.kt)

Program utama di `src/Main.kt` menguji dan mendemonstrasikan 14 skenario secara terstruktur mulai dari inisialisasi sistem, registrasi pengguna, pemesanan, demonstrasi dynamic dispatch, demonstrasi sealed class, pengujian enkapsulasi, hingga uji keamanan smart casting dan safe casting.

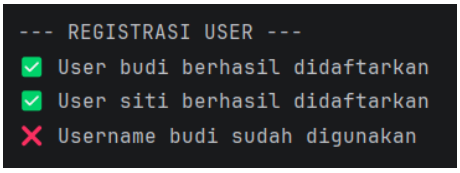
BAB IV
DEMONSTRASI DAN PENGUJIAN

4.1. Tampilan Output Terminal

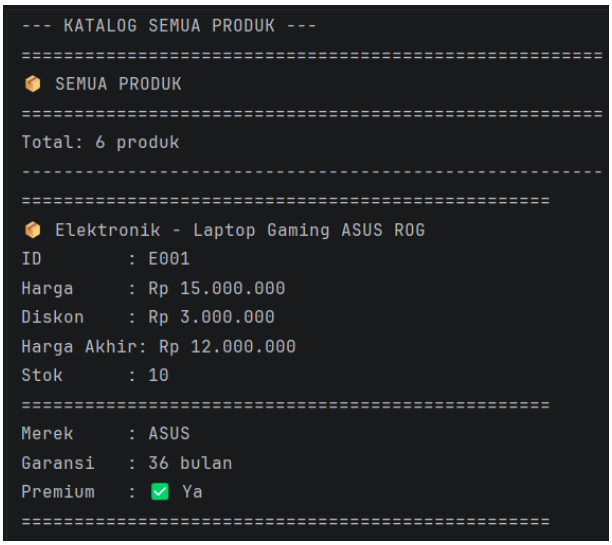
1) Banner & Inisialisasi Sistem serta Penambahan 6 Produk



2) Registrasi Pengguna dan Validasi Duplikasi Akun



3) Tampilan Katalog Produk Elektronik



```
================================================
📦 Elektronik - Smartphone Xiaomi Redmi
ID         : E002
Harga      : Rp 2.500.000
Diskon     : Rp 125.000
Harga Akhir: Rp 2.375.000
Stok       : 15
================================================
Merek      : Xiaomi
Garansi    : 12 bulan
Premium    : ❌ Tidak
================================================
```

4) Tampilan Katalog Produk Pakaian dan Makanan

```
================================================
📦 Pakaian - Kaos Polos Cotton Combed 30s
ID         : C002
Harga      : Rp 100.000
Diskon     : Rp 10.000
Harga Akhir: Rp 90.000
Stok       : 50
================================================
Ukuran     : L
Bahan      : Katun Combed
Seasonal   : ❌ Tidak
================================================
```

```
================================================
📦 Makanan - Beras Organik Raja Pandan
ID         : F001
Harga      : Rp 85.000
Diskon     : Rp 21.250
Harga Akhir: Rp 63.750
Stok       : 30
================================================
Berat      : 5000.0 gram
Kadaluarsa: 2026-12-31
Organik    : ✅ Ya
================================================
================================================
📦 Makanan - Mie Instan Goreng Spesial
ID         : F002
Harga      : Rp 3.500
Diskon     : Rp 175
Harga Akhir: Rp 3.325
Stok       : 100
================================================
Berat      : 85.0 gram
Kadaluarsa: 2026-06-30
Organik    : ❌ Tidak
================================================
```

5) Pengelolaan Keranjang Budi & Uji Batas Stok

```
--- BUDI MENAMBAH BARANG KE KERANJANG ---
✅ Laptop Gaming ASUS ROG x1 ditambahkan ke keranjang
✅ Jaket Musim Dingin Parka x2 ditambahkan ke keranjang
✅ Mie Instan Goreng Spesial x5 ditambahkan ke keranjang
Mencoba memesan barang melebihi stok yang tersedia:
❌ Stok tidak mencukupi (tersedia: 9)
```

6) Tampilan Rincian Keranjang Belanja Budi

```
--- TAMPILAN KERANJANG BELANJA BUDI ---
==================================================
🛒 KERANJANG BELANJA - budi
==================================================
Laptop Gaming ASUS ROG x1 = Rp 12.000.000
   (Diskon: Rp 3.000.000)
Jaket Musim Dingin Parka x2 = Rp 560.000
   (Diskon: Rp 240.000)
Mie Instan Goreng Spesial x5 = Rp 16.625
   (Diskon: Rp 875)
--------------------------------------------------
Total Diskon : Rp 3.240.875
Total Belanja: Rp 12.576.625
==================================================
```

7) Checkout Budi dengan Kartu Kredit

```
--- CHECKOUT BUDI (KARTU KREDIT) ---
Metode Pembayaran: Kartu Kredit
Biaya Layanan (Kartu Kredit): Rp 251.532
💳 Memproses pembayaran dengan Kartu Kredit...
✅ Pembayaran berhasil! ID: CC-1791282996762
✅ Order ORD-1791282996763 berhasil dibuat
```

8) Detail Order Budi dan Uji Transisi Status Pesanan

```
--- STATUS DAN DETAIL ORDER BUDI ---
==================================================
📋 RIWAYAT ORDER - budi
==================================================
1. Order ORD-1791282996763 - ⏳ Menunggu Pembayaran
   Total: Rp 12.576.625
Total Belanja: Rp 12.576.625
==================================================
```

```
======================================================
📋 DETAIL ORDER
======================================================
ID Order    : ORD-1791282996763
Pelanggan   : budi
Tanggal     : 2026-10-06T17:36:36.782111100
Status      : ⏳ Menunggu Pembayaran
------------------------------------------------------
Items:
   Laptop Gaming ASUS ROG x1 = Rp 12.000.000
   Jaket Musim Dingin Parka x2 = Rp 560.000
   Mie Instan Goreng Spesial x5 = Rp 16.625
------------------------------------------------------
Total Diskon: Rp 3.240.875
Total Harga : Rp 12.576.625
======================================================
```

```
>>> Demonstrasi Pembaruan Status Pesanan <<<
✅ Status order ORD-1791282996763 diubah menjadi: ✅ Dibayar
✅ Status order ORD-1791282996763 diubah menjadi: 🚚 Dikirim
✅ Status order ORD-1791282996763 diubah menjadi: 📦 Diterima
Mencoba mengubah status pesanan yang sudah berstatus final (Delivered):
❌ Status sudah final, tidak bisa diubah
```

9) Belanja dan Checkout Siti Menggunakan QRIS

```
--- SITI MENAMBAH BARANG KE KERANJANG ---
✅ Beras Organik Raja Pandan x2 ditambahkan ke keranjang
✅ Kaos Polos Cotton Combed 30s x3 ditambahkan ke keranjang
================================================
🛒 KERANJANG BELANJA - siti
================================================
Beras Organik Raja Pandan x2 = Rp 127.500
   (Diskon: Rp 42.500)
Kaos Polos Cotton Combed 30s x3 = Rp 270.000
   (Diskon: Rp 30.000)
------------------------------------------------
Total Diskon : Rp 72.500
Total Belanja: Rp 397.500
================================================

--- CHECKOUT SITI (QRIS) ---
Metode Pembayaran: QRIS
Biaya Layanan (QRIS): Rp 1.987
💳 Memproses pembayaran dengan QRIS...
✅ Pembayaran berhasil! ID: QR-1791282996790
✅ Order ORD-1791282996790 berhasil dibuat
✅ Status order ORD-1791282996790 diubah menjadi: ✅ Dibayar
✅ Status order ORD-1791282996790 diubah menjadi: 🚚 Dikirim
```

```
================================================
📋 RIWAYAT ORDER - siti
================================================
1. Order ORD-1791282996790 - 🚚 Dikirim
   Total: Rp 397.500
Total Belanja: Rp 397.500
================================================
```

10) Tampilan Laporan Penjualan Toko (Sales Report)

```
--- LAPORAN PENJUALAN TOKO KAMPUS ---
============================================================
📊 LAPORAN PENJUALAN
============================================================
Total Order   : 2
Total Revenue : Rp 12.974.125
------------------------------------------------------------
1. ORD-1791282996763 - budi
   Status: 📦 Diterima
   Total: Rp 12.576.625
2. ORD-1791282996790 - siti
   Status: 🚚 Dikirim
   Total: Rp 397.500
============================================================
```

11) Demonstrasi Polimorfisme (Dynamic Dispatch & Interface)

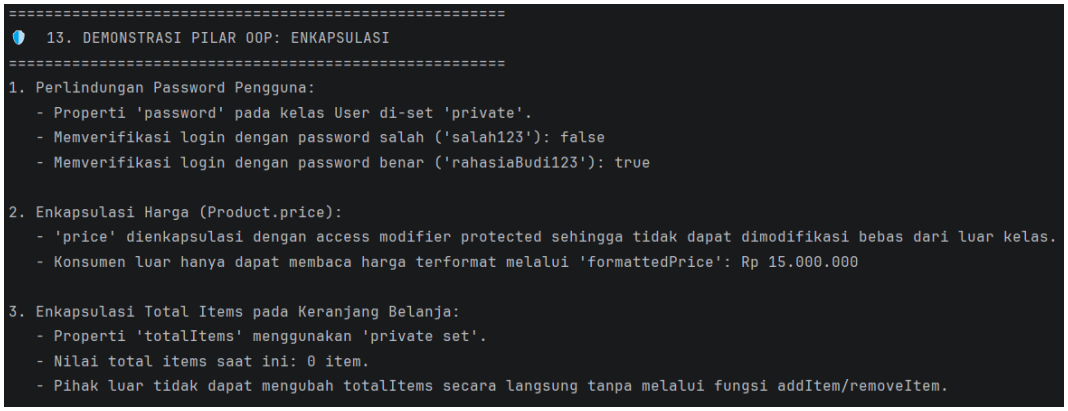
```
====================================================
🧠 11. DEMONSTRASI PILAR OOP: POLIMORFISME
====================================================
Menampung berbagai jenis subclass ke dalam List<Product> (Polymorphic Reference):
Produk: Laptop Gaming ASUS ROG
  -> Kategori  : Elektronik
  -> Harga Asli: Rp 15.000.000
  -> Diskon    : Rp 3.000.000
  -> Tagihan   : Rp 12.000.000
Produk: Jaket Musim Dingin Parka
  -> Kategori  : Pakaian
  -> Harga Asli: Rp 400.000
  -> Diskon    : Rp 120.000
  -> Tagihan   : Rp 280.000
Produk: Beras Organik Raja Pandan
  -> Kategori  : Makanan
  -> Harga Asli: Rp 85.000
  -> Diskon    : Rp 21.250
  -> Tagihan   : Rp 63.750

Polimorfisme pada gerbang pembayaran (PaymentMethod):
- Kartu Kredit: Fee untuk transaksi Rp 500.000 adalah Rp 10.000
- QRIS: Fee untuk transaksi Rp 500.000 adalah Rp 2.500
- Transfer Bank: Fee untuk transaksi Rp 500.000 adalah Rp 5.000
```

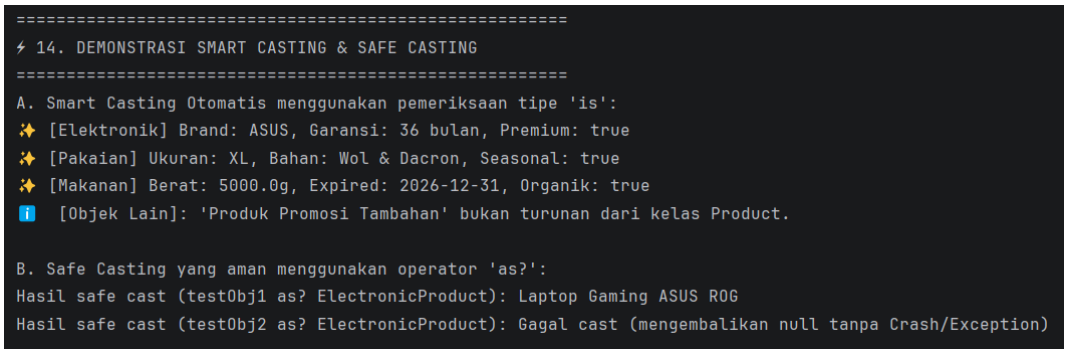
12) Demonstrasi Evaluasi Exhaustive When pada Sealed Class

```
====================================================
🔒 12. DEMONSTRASI SEALED CLASS (OrderStatus & PaymentResult)
====================================================
Evaluasi exhaustive 'when' pada sealed class OrderStatus:
Status [⏳ Menunggu Pembayaran] -> Tahap 1: Pembeli belum mentransfer dana. (Final? ❌)
Status [✅ Dibayar] -> Tahap 2: Dana sudah masuk, pesanan siap dipacking. (Final? ❌)
Status [🚚 Dikirim] -> Tahap 3: Paket sedang dalam perjalanan kurir. (Final? ❌)
Status [📦 Diterima] -> Tahap 4: Transaksi selesai, barang diterima. (Final? ✅)
Status [❌ Dibatalkan: Stok habis di gudang pusat] -> Tahap Pembatalan: Transaksi gugur (Alasan: Stok habis di gudang pusat). (Final? ✅)
```

13) Demonstrasi Enkapsulasi (Data Hiding & Access Protections)



14) Demonstrasi Smart Casting (is) dan Safe Casting (as?)



4.2. Analisis Skenario Pengujian Sistem

ID Uji	Skenario Pengujian	Input yang Diujikan	Hasil yang Diharapkan	Hasil Aktual	Kesimpulan
TC-01	Inisialisasi & Tambah Produk	6 objek dari 3 kategori berbeda	Seluruh produk masuk ke katalog	6 produk berhasil terdaftar	PASS
TC-02	Validasi Duplikasi User	Registrasi user "budi" dua kali	Registrasi kedua ditolak	Muncul pesan error username digunakan	PASS
TC-03	Pembatasan Stok Berlebih	Pesan 999 unit Laptop ASUS (stok 10)	Pembelian ditolak, stok tidak berkurang	Muncul peringatan stok tidak cukup	PASS
TC-04	Kalkulasi Diskon Keranjang	1 Laptop + 2 Jaket + 5 Mie Instan	Diskon dihitung tepat per kategori	Total diskon Rp 3.240.875 akurat	PASS

TC-05	Pembayaran Kartu Kredit	Kartu 16 digit valid & CVV 3 digit	Status pembayaran `Success`, terbit ID	`Success`, terbit `ID CC-...` & order	PASS
TC-06	Penguncian Status Final	Ubah order `Delivered` menjadi `Cancelled`	Perubahan status ditolak	Muncul pesan status final tidak bisa diubah	PASS
TC-07	Pembayaran QRIS	String QR Code ≥ 10 karakter	Transaksi berhasil dengan fee 0.5%	`Success`, fee Rp 1.987 terhitung akurat	PASS
TC-08	Laporan Omset Penjualan	Panggilan `displaySalesReport()`	Total order & omset terhitung akurat	2 Order, Revenue Rp 12.974.125	PASS
TC-09	Polimorfisme Dynamic Dispatch	Iterasi `List<Product>` panggil `calculateDiscount()`	Tiap subclass menjalankan formulanya	Diskon ASUS 20%, Jaket 30%, Beras 25%	PASS
TC-10	Keamanan Password	Uji `user.authenticate("salah")`	Mengembalikan nilai `false`	Mengembalikan `false` tanpa leak password	PASS
TC-11	Safe Casting Kotlin	`testObj as? ElectronicProduct` pada String	Mengembalikan `null` tanpa crash	Mengembalikan `null` aman	PASS

BAB V
ANALISIS PENERAPAN PILAR OOP

5.1. Enkapsulasi (Encapsulation)

Prinsip enkapsulasi diterapkan untuk melindungi integritas internal objek dari modifikasi luar yang tidak sah (*data hiding*):

1) Akses Modifier `protected` pada `Product.price`:

```
abstract class Product(
    val id: String,
    val name: String,
    protected var price: Double,
    var stock: Int
) {
    val formattedPrice: String
        get() = "Rp ${formatRupiah(price)}"
    ...
}
```

Analisis: Variabel `price` tidak dapat diubah seenaknya oleh kelas luar (misal kode di `Main.kt` tidak dapat melakukan `product.price = 0.0`). Namun, modifier `protected` mengizinkan subclass turunan seperti `ElectronicProduct` membaca nilai `price` untuk mengalikan persentase diskon. Akses pembacaan publik difasilitasi melalui getter terformat `formattedPrice`.

2) Penyembunyian Password pada Kelas `User`:

```
class User(
    val username: String,
    val email: String,
    private val password: String
) {
    fun authenticate(inputPassword: String): Boolean {
        return password == inputPassword
    }
}
```

Analisis: Properti `password` dideklarasikan `private` tanpa properti getter. Dengan demikian, sistem luar sama sekali tidak memiliki jalan untuk membaca kata sandi mentah. Autentikasi hanya dapat dilakukan secara tertutup melalui fungsi boolean `authenticate()`.

3) Private Setter pada `ShoppingCart.totalItems`:

```
class ShoppingCart(val owner: String) {
    private val items = mutableMapOf<Product, Int>()
    var totalItems: Int = 0
        private set
    ...
}
```

Analisis: Properti `totalItems` memiliki getter publik sehingga siapa pun dapat memeriksa jumlah barang di keranjang, tetapi setter-nya dilindungi secara privat (`private set`). Nilai `totalItems` hanya dapat berubah jika kuantitas barang divalidasi dan stok mencukupi melalui metode `addItem()` atau `removeItem()`.

5.2. Pewarisan (Inheritance)

Prinsip pewarisan dimanfaatkan untuk menciptakan hierarki taksonomi produk yang rapi dan menghindari duplikasi kode:

```
class ElectronicProduct(
    id: String,
    name: String,
    price: Double,
    stock: Int,
    val brand: String,
    val warrantyMonths: Int,
    val isPremium: Boolean
) : Product(id, name, price, stock) {

    override fun calculateDiscount(): Double {
        var discount = 0.0
        discount += price * 0.05
        if (isPremium) discount += price * 0.10
        if (warrantyMonths > 24) discount += price * 0.05
        return discount
```

```
    }

    override fun getCategory(): String = "Elektronik"

    override fun displayInfo() {
        super.displayInfo()
        println("Merek     : $brand")
        println("Garansi   : $warrantyMonths bulan")
        println("Premium   : ${if (isPremium) "✅ Ya" else "❌
Tidak"}")
        println("=".repeat(50))
    }
}
```

Analisis:

- `ElectronicProduct`, `ClothingProduct`, dan `FoodProduct` mewarisi properti dasar (`id`, `name`, `price`, `stock`) dan metode beton (`reduceStock`, `getDiscountedPrice`) dari kelas induk `Product`.
- Kata kunci `super.displayInfo()` mendemonstrasikan pemanfaatan kembali fungsionalitas pencetakan informasi umum kelas induk sebelum subclass menambahkan atribut khususnya masing-masing.

5.3. Polimorfisme (Polymorphism)

Polimorfisme memungkinkan pemrosesan berbagai varian objek melalui satu antarmuka yang seragam:

1) Dynamic Method Dispatch pada Koleksi Produk:

```
val polymorphicProductList: List<Product> = listOf(laptopAsus,
jaketWinter, berasOrganik)

polymorphicProductList.forEach { product ->
    val diskon = product.calculateDiscount()
    println("${product.name} diskon: Rp $diskon")
}
```

Analisis: Walaupun seluruh elemen ditampung dalam tipe koleksi generik `List<Product>`, pada saat metode `calculateDiscount()` dipanggil, mesin

JVM secara dinamis mengarahkan eksekusi ke metode override milik subclass yang bersangkutan (apakah itu diskon elektronik, diskon musiman pakaian, atau diskon organik).

2) **Polimorfisme Antarmuka Pembayaran:**

```
fun checkout(paymentMethod: PaymentMethod): Order? {
    val amount = cart.getTotalPrice()
    val result = paymentMethod.processPayment(amount)
    ...
}
```

Analisis: Fungsi `checkout` pada kelas `User` tidak terikat (*loosely coupled*) pada implementasi spesifik pembayaran. Objek apa pun yang memenuhi kontrak interface `PaymentMethod` (baik `CreditCardPayment`, `QRISPayment`, maupun `BankTransferPayment`) dapat diterima dan diproses secara mulus.

5.4. Abstraksi (Abstraction)

Abstraksi memisahkan antara konsep antarmuka dengan detail operasionalnya:

1) **Abstract Class `Product`:** Mendefinisikan esensi konseptual bahwa setiap produk di toko wajib memiliki mekanisme penghitungan diskon (`calculateDiscount()`) dan penamaan kategori (`getCategory()`), namun cara penghitungan diserahkan sepenuhnya kepada subclass spesifik.
2) **Interface `PaymentMethod`:** Mengabstraksikan transaksi perbankan dan gerbang pembayaran menjadi dua operasi esensial: memproses pembayaran (`processPayment`) dan menghitung tarif transaksi (`getFee`).

5.5. Fitur Lanjutan Kotlin: Sealed Class & Smart Casting

1) **Sealed Class `OrderStatus`:**

```
sealed class OrderStatus {
    data object Pending : OrderStatus()
    data object Paid : OrderStatus()
    data object Shipped : OrderStatus()
    data object Delivered : OrderStatus()
    data class Cancelled(val reason: String) : OrderStatus()
    abstract fun display(): String
```

```
    fun isFinal(): Boolean = this is Delivered || this is
Cancelled
}
```

Analisis: Berbeda dengan kelas enum biasa yang hanya menyimpan konstanta statis, subclass dari sealed class dapat menyimpan data dinamis (seperti atribut `reason` pada `Cancelled`). Selain itu, ekspresi `when (status)` dapat dipastikan ekshaustif (*exhaustive*) tanpa memerlukan branch `else`, sehingga jika di masa depan ditambahkan status baru, kompiler akan langsung memberikan peringatan error di seluruh bagian kode yang belum menangani status tersebut.

2) Smart Casting Otomatis:

```
when (item) {
    is ElectronicProduct -> {
        // 'item' otomatis di-smart cast ke ElectronicProduct
        println("Brand: ${item.brand}, Garansi:
${item.warrantyMonths}")
    }
    is ClothingProduct -> {
        // 'item' otomatis di-smart cast ke ClothingProduct
        println("Ukuran: ${item.size}, Bahan:
${item.material}")
    }
}
```

Analisis: Kompiler Kotlin menganalisis alur kode (*flow analysis*). Begitu tipe diperiksa menggunakan operator `is`, referensi `item` otomatis dianggap sebagai subclass tersebut sehingga pengembang tidak perlu melakukan *type-casting* eksplisit yang rentan kesalahan.

BAB VI
KESIMPULAN DAN SARAN

6.1. Kesimpulan

Berdasarkan perancangan, implementasi, dan pengujian yang telah dilakukan, dapat diambil kesimpulan sebagai berikut:

1) Empat Pilar OOP Berhasil Diterapkan Secara Komprehensif: Enkapsulasi berhasil mengamankan kata sandi dan harga; Pewarisan berhasil membentuk hierarki produk yang bersih; Polimorfisme berhasil menyatukan pemrosesan multi-produk dan multi-pembayaran; serta Abstraksi berhasil menyusun cetak biru sistem melalui abstract class dan interface.
2) Karakteristik Modern Kotlin Meningkatkan Kualitas Kode: Pemanfaatan *Sealed Class* memberikan keamanan tipe tingkat tinggi dalam siklus hidup pesanan, sementara *Smart Casting* dan *Safe Casting* mengeliminasi potensi galat *runtime* seperti `ClassCastException`.
3) Sistem Teruji Bebas Kesalahan: Pengujian seluruh 14 skenario membuktikan bahwa seluruh logika bisnis, mulai dari pengurangan stok otomatis, penolakan pesanan saat stok habis, perhitungan diskon multi-kategori, kalkulasi biaya transaksi, hingga pemblokiran manipulasi order final berjalan 100% sesuai spesifikasi kebutuhan.

6.2. Saran

Untuk pengembangan sistem lebih lanjut, disarankan beberapa peningkatan berikut:

1) Integrasi Basis Data Persisten: Menggantikan koleksi in-memory (`MutableList` dan `MutableMap`) dengan basis data relasional (seperti PostgreSQL/SQLite) atau ORM (Exposed/JPA) agar data tidak hilang saat aplikasi ditutup.
2) Antarmuka Pengguna Visual (GUI / Web API): Mengembangkan antarmuka berbasis web menggunakan framework Ktor / Spring Boot atau antarmuka desktop menggunakan Compose for Desktop.
3) Pengelolaan Kupon Promosi Dinamis: Menambahkan modul kupon diskon global yang dapat dikombinasikan dengan diskon bawaan produk.

BAB VII

LAMPIRAN

7.1. File `src/product/Product.kt`

```
package product

/**
 * Kelas abstrak yang merepresentasikan produk umum di dalam sistem
e-commerce.
 *
 * Menerapkan prinsip **Abstraksi** dan **Enkapsulasi**:
 * - Properti [price] dienkapsulasi dengan modifier `protected`
sehingga hanya dapat diakses dalam hierarki inheritance [Product].
 * - Menyediakan properti terhitung [formattedPrice] untuk
mendapatkan harga dalam format Rupiah yang rapi.
 * - Mendefinisikan metode abstrak [calculateDiscount] dan
[getCategory] yang wajib diimplementasikan oleh setiap subclass.
 *
 * @property id ID unik produk (contoh: "E001", "C001")
 * @property name Nama produk
 * @property price Nilai harga dasar produk (protected untuk
enkapsulasi)
 * @property stock Jumlah stok produk yang tersedia di inventaris
 */
abstract class Product(
    val id: String,
    val name: String,
    protected var price: Double,
    var stock: Int
) {
    // ============================================================
    // PROPERTI
    // ============================================================

    /**
     * Getter untuk harga produk dalam format mata uang Rupiah.
     */
    val formattedPrice: String
        get() = "Rp ${formatRupiah(price)}"

    // ============================================================
    // METODE ABSTRAK
    // ============================================================

    /**
     * Menghitung diskon yang berlaku untuk produk.
     * Implementasi besaran diskon berbeda-beda pada setiap
subclass.
     *
     * @return Jumlah potongan diskon dalam satuan Rupiah
     */
    abstract fun calculateDiscount(): Double

    /**
     * Mendapatkan nama kategori produk.
     *
     * @return Nama kategori produk (misal: "Elektronik", "Pakaian",
"Makanan")
```

```
     */
    abstract fun getCategory(): String

    // ============================================================
    // METODE CONCRETE (Dapat digunakan oleh semua subclass)
    // ============================================================

    /**
     * Menghitung harga akhir setelah dipotong diskon.
     *
     * @return Harga akhir dalam satuan Rupiah
     */
    open fun getDiscountedPrice(): Double {
        return price - calculateDiscount()
    }

    /**
     * Menampilkan informasi detail produk ke konsol.
     * Dapat di-override oleh subclass untuk menyertakan atribut
spesifik kategori.
     */
    open fun displayInfo() {
        println("=".repeat(50))
        println("📦 ${getCategory()} - $name")
        println("ID        : $id")
        println("Harga     : $formattedPrice")
        println("Diskon    : Rp
${formatRupiah(calculateDiscount())}")
        println("Harga Akhir: Rp
${formatRupiah(getDiscountedPrice())}")
        println("Stok      : $stock")
        println("=".repeat(50))
    }

    /**
     * Mengurangi jumlah stok produk jika stok mencukupi.
     * Dapat juga menerima nilai negatif untuk mengembalikan stok.
     *
     * @param quantity Jumlah unit produk yang dikurangi
     * @return `true` jika pengurangan stok berhasil, `false` jika
stok tidak mencukupi
     */
    fun reduceStock(quantity: Int): Boolean {
        return if (stock >= quantity) {
            stock -= quantity
            true
        } else {
            false
        }
    }

    // ============================================================
    // HELPER METHOD (Protected)
    // ============================================================

    /**
     * Metode pembantu untuk memformat nilai desimal uang menjadi
format mata uang Rupiah
     * dengan pemisah ribuan menggunakan titik.
     *
     * @param nominal Angka nominal uang dalam Double
```

```
     * @return String angka terformat (contoh: "15.000.000")
     */
    fun formatRupiah(nominal: Double): String {
        val str = nominal.toLong().toString()
        val builder = StringBuilder()
        var count = 0
        for (i in str.length - 1 downTo 0) {
            builder.insert(0, str[i])
            count++
            if (count % 3 == 0 && i > 0) {
                builder.insert(0, ".")
            }
        }
        return builder.toString()
    }
}
```

7.2. File `src/product/ElectronicProduct.kt`

```
package product

/**
 * Representasi produk kategori Elektronik di dalam sistem e-
commerce.
 *
 * Menerapkan prinsip **Pewarisan** (Inheritance) dari kelas
[Product]:
 * - Menambahkan atribut khusus [brand], [warrantyMonths], dan
[isPremium].
 * - Meng-override [calculateDiscount] dengan aturan bisnis khusus
elektronik.
 * - Meng-override [displayInfo] dengan memanfaatkan kata kunci
`super` untuk memanggil implementasi kelas induk.
 *
 * @param id ID unik produk
 * @param name Nama produk elektronik
 * @param price Nilai harga dasar produk
 * @param stock Jumlah unit produk dalam inventaris
 * @property brand Merek atau pabrikan barang elektronik (contoh:
"ASUS", "Samsung")
 * @property warrantyMonths Durasi masa garansi dalam satuan bulan
 * @property isPremium Menandakan apakah produk termasuk kategori
premium yang berhak atas diskon tambahan
 */
class ElectronicProduct(
    id: String,
    name: String,
    price: Double,
    stock: Int,
    val brand: String,
    val warrantyMonths: Int,
    val isPremium: Boolean
) : Product(id, name, price, stock) {

    /**
     * Menghitung besaran diskon khusus produk elektronik:
     * - Diskon dasar: 5% untuk seluruh barang elektronik.
     * - Diskon tambahan: 10% jika produk berstatus [isPremium].
```

```
     * - Diskon tambahan: 5% jika masa garansi lebih dari 24 bulan
([warrantyMonths] > 24).
     *
     * @return Total potongan diskon dalam satuan Rupiah
     */
    override fun calculateDiscount(): Double {
        var discount = 0.0

        // Diskon dasar: 5% untuk semua elektronik
        discount += price * 0.05

        // Diskon tambahan: 10% jika premium
        if (isPremium) {
            discount += price * 0.10
        }

        // Diskon tambahan: 5% jika garansi > 24 bulan
        if (warrantyMonths > 24) {
            discount += price * 0.05
        }

        return discount
    }

    /**
     * Mengembalikan nama kategori produk yaitu "Elektronik".
     */
    override fun getCategory(): String = "Elektronik"

    /**
     * Menampilkan informasi produk secara lengkap termasuk
spesifikasi khusus elektronik:
     * merek, durasi garansi, dan status premium.
     */
    override fun displayInfo() {
        super.displayInfo()
        println("Merek     : $brand")
        println("Garansi   : $warrantyMonths bulan")
        println("Premium   : ${if (isPremium) "✅ Ya" else "❌
Tidak"}")
        println("=".repeat(50))
    }
}
```

7.3. File `src/product/ClothingProduct.kt`

```
package product

/**
 * Representasi produk kategori Pakaian di dalam sistem e-commerce.
 *
 * Menerapkan prinsip **Pewarisan** (Inheritance) dari kelas
[Product]:
 * - Menambahkan atribut spesifik [size], [material], dan
[isSeasonal].
 * - Meng-override [calculateDiscount] dengan skema diskon busana
(diskon dasar, seasonal, dan ukuran besar).
 * - Meng-override [displayInfo] untuk menampilkan atribut pakaian.
 *
```

```
 * @param id ID unik produk
 * @param name Nama produk pakaian
 * @param price Nilai harga dasar produk
 * @param stock Jumlah unit produk dalam inventaris
 * @property size Ukuran pakaian (contoh: "S", "M", "L", "XL",
"XXL")
 * @property material Bahan pembuat pakaian (contoh: "Katun", "Wol",
"Polyester")
 * @property isSeasonal Menandakan apakah pakaian bersifat musiman
(misal: pakaian musim dingin) yang memperoleh diskon khusus
 */
class ClothingProduct(
    id: String,
    name: String,
    price: Double,
    stock: Int,
    val size: String,
    val material: String,
    val isSeasonal: Boolean
) : Product(id, name, price, stock) {

    /**
     * Menghitung diskon khusus produk pakaian:
     * - Diskon dasar: 10% untuk semua produk pakaian.
     * - Diskon tambahan: 15% jika produk bersifat seasonal
([isSeasonal]).
     * - Diskon tambahan: 5% jika ukuran pakaian adalah XL, XXL,
atau XXXL.
     *
     * @return Total potongan diskon dalam satuan Rupiah
     */
    override fun calculateDiscount(): Double {
        var discount = 0.0

        // Diskon dasar: 10% untuk semua pakaian
        discount += price * 0.10

        // Diskon tambahan: 15% jika seasonal
        if (isSeasonal) {
            discount += price * 0.15
        }

        // Diskon tambahan: 5% untuk size XL atau lebih besar
        if (size.uppercase() in listOf("XL", "XXL", "XXXL")) {
            discount += price * 0.05
        }

        return discount
    }

    /**
     * Mengembalikan nama kategori produk yaitu "Pakaian".
     */
    override fun getCategory(): String = "Pakaian"

    /**
     * Menampilkan informasi produk secara lengkap termasuk ukuran
pakaian, bahan, dan status musiman.
     */
    override fun displayInfo() {
        super.displayInfo()
```

```
        println("Ukuran    : $size")
        println("Bahan     : $material")
        println("Seasonal  : ${if (isSeasonal) "✅ Ya" else "❌
Tidak"}")
        println("=".repeat(50))
    }
}
```

7.4. File `src/product/FoodProduct.kt`

```
package product

/**
 * Representasi produk kategori Makanan di dalam sistem e-commerce.
 *
 * Menerapkan prinsip **Pewarisan** (Inheritance) dari kelas
[Product]:
 * - Menambahkan atribut khusus makanan [expiryDate], [weight], dan
[isOrganic].
 * - Meng-override [calculateDiscount] dengan diskon dasar dan
diskon makanan organik.
 * - Meng-override [displayInfo] untuk menampilkan atribut makanan.
 *
 * @param id ID unik produk
 * @param name Nama produk makanan
 * @param price Nilai harga dasar produk
 * @param stock Jumlah unit produk dalam inventaris
 * @property expiryDate Tanggal kadaluarsa produk dalam format
tanggal (contoh: "2026-12-31")
 * @property weight Berat bersih produk dalam satuan gram (contoh:
5000.0)
 * @property isOrganic Menandakan apakah produk merupakan makanan
organik bersertifikasi yang berhak atas diskon khusus
 */
class FoodProduct(
    id: String,
    name: String,
    price: Double,
    stock: Int,
    val expiryDate: String,
    val weight: Double,
    val isOrganic: Boolean
) : Product(id, name, price, stock) {

    /**
     * Menghitung diskon khusus produk makanan:
     * - Diskon dasar: 5% untuk semua kategori makanan.
     * - Diskon tambahan: 20% jika produk merupakan produk organik
([isOrganic]).
     *
     * @return Total potongan diskon dalam satuan Rupiah
     */
    override fun calculateDiscount(): Double {
        var discount = 0.0

        // Diskon dasar: 5% untuk semua makanan
        discount += price * 0.05

        // Diskon tambahan: 20% jika organik
```

```
        if (isOrganic) {
            discount += price * 0.20
        }

        return discount
    }

    /**
     * Mengembalikan nama kategori produk yaitu "Makanan".
     */
    override fun getCategory(): String = "Makanan"

    /**
     * Menampilkan informasi produk secara lengkap termasuk berat
netto, tanggal kadaluarsa, dan status organik.
     */
    override fun displayInfo() {
        super.displayInfo()
        println("Berat     : $weight gram")
        println("Kadaluarsa: $expiryDate")
        println("Organik   : ${if (isOrganic) "✅ Ya" else "❌
Tidak"}")
        println("=".repeat(50))
    }
}
```

7.5. File `src/cart/ShoppingCart.kt`

```
package cart

import product.Product

/**
 * Kelas yang merepresentasikan keranjang belanja seorang pelanggan.
 *
 * Menerapkan prinsip **Enkapsulasi**:
 * - Koleksi [items] bersifat `private` untuk mencegah manipulasi
eksternal tanpa validasi stok.
 * - Properti [totalItems] memiliki getter publik namun setter
`private` ([private set]),
 *   sehingga hanya metode internal yang dapat memodifikasi jumlah
item.
 *
 * @property owner Nama pemilik keranjang belanja
 */
class ShoppingCart(val owner: String) {

    // ============================================================
    // PROPERTI
    // ============================================================

    /**
     * Map privat yang memetakan objek [Product] ke jumlah kuantitas
yang dimasukkan ke keranjang.
     */
    private val items = mutableMapOf<Product, Int>()

    /**
     * Total jumlah kuantitas barang yang ada di dalam keranjang.
```

```
     * Hanya dapat dibaca secara publik, mutasi hanya melalui metode
[addItem], [removeItem], atau [clear].
     */
    var totalItems: Int = 0
        private set

    // ============================================================
    // METODE
    // ============================================================

    /**
     * Menambahkan sejumlah unit produk ke dalam keranjang.
     * Melakukan validasi kuantitas harus positif dan memeriksa
ketersediaan stok pada [product].
     *
     * @param product Objek produk yang akan ditambahkan
     * @param quantity Jumlah unit produk
     * @return `true` jika berhasil ditambahkan dan stok mencukupi,
`false` jika gagal
     */
    fun addItem(product: Product, quantity: Int): Boolean {
        if (quantity <= 0) {
            println("❌ Jumlah harus lebih dari 0")
            return false
        }

        if (!product.reduceStock(quantity)) {
            println("❌ Stok tidak mencukupi (tersedia:
${product.stock})")
            return false
        }

        items[product] = items.getOrDefault(product, 0) + quantity
        totalItems += quantity
        println("✅ ${product.name} x$quantity ditambahkan ke
keranjang")
        return true
    }

    /**
     * Menghapus seluruh kuantitas suatu produk dari keranjang
belanja
     * dan mengembalikan stok produk tersebut ke inventaris.
     *
     * @param product Objek produk yang akan dihapus dari keranjang
     * @return `true` jika produk ada di keranjang dan berhasil
dihapus, `false` jika tidak ditemukan
     */
    fun removeItem(product: Product): Boolean {
        val quantity = items[product] ?: return false
        items.remove(product)
        totalItems -= quantity
        // Kembalikan stok ke produk
        product.reduceStock(-quantity)
        println("✅ ${product.name} dihapus dari keranjang")
        return true
    }

    /**
```

```
     * Menghitung total harga belanja seluruh barang di keranjang
setelah dipotong diskon masing-masing produk.
     *
     * @return Total biaya belanja dalam satuan Rupiah
     */
    fun getTotalPrice(): Double {
        return items.entries.sumOf { (product, quantity) ->
            product.getDiscountedPrice() * quantity
        }
    }

    /**
     * Menghitung total akumulasi penghematan diskon dari seluruh
barang di keranjang.
     *
     * @return Total nominal diskon dalam satuan Rupiah
     */
    fun getTotalDiscount(): Double {
        return items.entries.sumOf { (product, quantity) ->
            product.calculateDiscount() * quantity
        }
    }

    /**
     * Mendapatkan salinan read-only dari seluruh item produk
beserta kuantitasnya yang ada di keranjang.
     *
     * @return Map immutable berisi [Product] dan kuantitasnya
     */
    fun getItems(): Map<Product, Int> = items.toMap()

    /**
     * Memeriksa apakah keranjang belanja saat ini dalam keadaan
kosong.
     *
     * @return `true` jika tidak ada barang di keranjang, `false`
jika ada
     */
    fun isEmpty(): Boolean = items.isEmpty()

    /**
     * Mengosongkan seluruh item dari keranjang belanja dan mereset
[totalItems] ke 0.
     * Digunakan setelah proses checkout pesanan berhasil
diselesaikan.
     */
    fun clear() {
        items.clear()
        totalItems = 0
    }

    /**
     * Menampilkan rincian isi keranjang belanja ke konsol, termasuk
nama barang,
     * kuantitas, harga subtotal, potongan diskon, dan total
keseluruhan.
     */
    fun displayCart() {
        println("=".repeat(50))
        println("🛒 KERANJANG BELANJA - $owner")
        println("=".repeat(50))
```

```
        if (items.isEmpty()) {
            println("   Keranjang kosong")
        } else {
            items.forEach { (product, quantity) ->
                println("${product.name} x$quantity = Rp
${formatRupiah(product.getDiscountedPrice() * quantity)}")
                println("   (Diskon: Rp
${formatRupiah(product.calculateDiscount() * quantity)})")
            }
            println("-".repeat(50))
            println("Total Diskon : Rp
${formatRupiah(getTotalDiscount())}")
            println("Total Belanja: Rp
${formatRupiah(getTotalPrice())}")
        }
        println("=".repeat(50))
    }

    /**
     * Helper privat untuk memformat nominal angka menjadi format
Rupiah dengan pemisah ribuan titik.
     */
    private fun formatRupiah(nominal: Double): String {
        val str = nominal.toLong().toString()
        val builder = StringBuilder()
        var count = 0
        for (i in str.length - 1 downTo 0) {
            builder.insert(0, str[i])
            count++
            if (count % 3 == 0 && i > 0) {
                builder.insert(0, ".")
            }
        }
        return builder.toString()
    }
}
```

7.6. File `src/order/OrderStatus.kt`

```
package order

/**
 * Representasi status pesanan dalam sistem e-commerce menggunakan
**Sealed Class**.
 *
 * Menerapkan konsep **Sealed Class** dan **Polimorfisme**:
 * - Membatasi hierarki status yang valid hanya pada jenis status
yang telah didefinisikan secara terbatas.
 * - Memungkinkan pengecekan `when` secara ekshaustif (exhaustive
check) tanpa memerlukan branch `else`.
 * - Menyediakan metode abstrak [display] dan helper [isFinal] untuk
menentukan apakah status sudah berstatus akhir.
 */
sealed class OrderStatus {

    /**
     * Status ketika pesanan baru dibuat dan menunggu pelunasan
pembayaran.
```

```
     */
    data object Pending : OrderStatus() {
        override fun display(): String = "⏳ Menunggu Pembayaran"
    }

    /**
     * Status ketika pembayaran pesanan telah berhasil diverifikasi.
     */
    data object Paid : OrderStatus() {
        override fun display(): String = "✅ Dibayar"
    }

    /**
     * Status ketika barang pesanan sedang dalam proses pengiriman
oleh kurir logistik.
     */
    data object Shipped : OrderStatus() {
        override fun display(): String = "🚚 Dikirim"
    }

    /**
     * Status ketika pesanan telah tiba di tujuan dan diterima oleh
pelanggan (status final).
     */
    data object Delivered : OrderStatus() {
        override fun display(): String = "📦 Diterima"
    }

    /**
     * Status ketika pesanan dibatalkan beserta alasannya (status
final).
     *
     * @property reason Alasan pembatalan pesanan
     */
    data class Cancelled(val reason: String) : OrderStatus() {
        override fun display(): String = "❌ Dibatalkan: $reason"
    }

    /**
     * Mengembalikan teks representasi status pesanan lengkap dengan
ikon indikator.
     *
     * @return String representasi status pesanan
     */
    abstract fun display(): String

    /**
     * Memeriksa apakah status pesanan sudah bersifat final
([Delivered] atau [Cancelled]).
     * Pesanan dengan status final tidak dapat diubah lagi
statusnya.
     *
     * @return `true` jika status adalah Delivered atau Cancelled,
`false` jika masih dalam proses
     */
    fun isFinal(): Boolean {
        return this is Delivered || this is Cancelled
    }
}
```

7.7. File `src/order/Order.kt`

```
package order

import product.Product

/**
 * Kelas yang merepresentasikan pesanan pembelian (order) di dalam
sistem e-commerce.
 *
 * Menerapkan prinsip **Enkapsulasi** dan **Polimorfisme**:
 * - Menyimpan snapshot daftar produk yang dibeli melalui [items].
 * - Menghitung total harga dan total diskon secara dinamis melalui
properti terhitung.
 * - Menggunakan [OrderStatus] (sealed class) untuk melacak status
transaksi dengan proteksi [updateStatus]
 *   sehingga pesanan yang sudah berstatus final tidak dapat diubah
kembali.
 *
 * @property id ID unik pesanan (contoh: "ORD-1700000000001")
 * @property customerName Nama pelanggan yang melakukan pemesanan
 * @property items Map berisi daftar produk dan kuantitas yang
dipesan
 * @property status Status pesanan saat ini, default bernilai
[OrderStatus.Pending]
 */
class Order(
    val id: String,
    val customerName: String,
    val items: Map<Product, Int>,
    var status: OrderStatus = OrderStatus.Pending
) {
    // ============================================================
    // PROPERTI
    // ============================================================

    /**
     * Total nilai pembayaran pesanan setelah memperhitungkan diskon
tiap produk.
     */
    val totalPrice: Double
        get() = items.entries.sumOf { (product, quantity) ->
            product.getDiscountedPrice() * quantity
        }

    /**
     * Total potongan diskon yang didapatkan pelanggan pada pesanan
ini.
     */
    val totalDiscount: Double
        get() = items.entries.sumOf { (product, quantity) ->
            product.calculateDiscount() * quantity
        }

    /**
     * Waktu penciptaan pesanan dalam format ISO LocalDateTime.
     */
    val createdAt: String = java.time.LocalDateTime.now().toString()
```

```
    // ============================================================
    // METODE
    // ============================================================

    /**
     * Memperbarui status pesanan ke status baru.
     * Mengembalikan nilai `false` dan menolak perubahan jika status
pesanan saat ini telah final ([OrderStatus.isFinal]).
     *
     * @param newStatus Status pesanan baru yang akan diaplikasikan
     * @return `true` jika status berhasil diperbarui, `false` jika
pesanan sudah dalam status final
     */
    fun updateStatus(newStatus: OrderStatus): Boolean {
        if (status.isFinal()) {
            println("❌ Status sudah final, tidak bisa diubah")
            return false
        }
        status = newStatus
        println("✅ Status order $id diubah menjadi:
${status.display()}")
        return true
    }

    /**
     * Menampilkan informasi rincian nota pesanan ke konsol.
     */
    fun displayOrder() {
        println("=".repeat(55))
        println("📋 DETAIL ORDER")
        println("=".repeat(55))
        println("ID Order    : $id")
        println("Pelanggan   : $customerName")
        println("Tanggal     : $createdAt")
        println("Status      : ${status.display()}")
        println("-".repeat(55))
        println("Items:")
        items.forEach { (product, quantity) ->
            println("   ${product.name} x$quantity = Rp
${formatRupiah(product.getDiscountedPrice() * quantity)}")
        }
        println("-".repeat(55))
        println("Total Diskon: Rp ${formatRupiah(totalDiscount)}")
        println("Total Harga : Rp ${formatRupiah(totalPrice)}")
        println("=".repeat(55))
    }

    /**
     * Helper privat untuk memformat angka nominal menjadi format
Rupiah dengan pemisah ribuan.
     */
    private fun formatRupiah(nominal: Double): String {
        val str = nominal.toLong().toString()
        val builder = StringBuilder()
        var count = 0
        for (i in str.length - 1 downTo 0) {
            builder.insert(0, str[i])
            count++
            if (count % 3 == 0 && i > 0) {
                builder.insert(0, ".")
```

```
            }
        }
        return builder.toString()
    }
}
```

7.8. File `src/payment/PaymentMethod.kt`

```
package payment

/**
 * Interface yang mendefinisikan kontrak sistem pembayaran di e-
commerce.
 *
 * Menerapkan prinsip **Abstraksi** dan **Polimorfisme**:
 * - Mengabstraksikan perilaku berbagai macam gerbang pembayaran
(Payment Gateway).
 * - Kelas yang mengimplementasikan interface ini wajib menyediakan
kalkulasi biaya ([getFee])
 *   dan mekanisme pemrosesan transaksi ([processPayment]).
 */
interface PaymentMethod {

    /**
     * Nama metode pembayaran (misal: "Kartu Kredit", "QRIS",
"Transfer Bank").
     */
    val name: String

    /**
     * Memproses transaksi pembayaran dengan nominal tertentu.
     *
     * @param amount Nominal transaksi yang harus dibayarkan
     * @return Objek [PaymentResult] yang merepresentasikan hasil
proses pembayaran
     */
    fun processPayment(amount: Double): PaymentResult

    /**
     * Menghitung biaya administrasi/transaksi berdasarkan nominal
pembayaran.
     *
     * @param amount Nominal transaksi
     * @return Besaran biaya transaksi dalam Rupiah
     */
    fun getFee(amount: Double): Double
}

/**
 * Representasi status hasil pemrosesan transaksi pembayaran
menggunakan **Sealed Class**.
 *
 * Memungkinkan penanganan hasil pembayaran secara komprehensif dan
type-safe
 * melalui percabangan `when`.
 */
sealed class PaymentResult {

    /**
```

```
     * Status ketika pembayaran sukses diverifikasi.
     *
     * @property transactionId Kode unik referensi transaksi yang
berhasil dibuat
     */
    data class Success(val transactionId: String) : PaymentResult()

    /**
     * Status ketika pembayaran gagal diproses karena kesalahan
input atau penolakan bank.
     *
     * @property reason Keterangan penyebab kegagalan transaksi
     * @property errorCode Kode status error numerik
     */
    data class Failed(val reason: String, val errorCode: Int) :
PaymentResult()

    /**
     * Status ketika transaksi sedang menunggu konfirmasi atau
verifikasi pihak ketiga.
     */
    data object Pending : PaymentResult()
}
```

7.9. File `src/payment/CreditCardPayment.kt`

```
package payment

/**
 * Implementasi metode pembayaran menggunakan Kartu Kredit.
 *
 * Menerapkan prinsip **Abstraksi** dan **Enkapsulasi**:
 * - Mengimplementasikan kontrak [PaymentMethod].
 * - Data sensitif kartu kredit ([cardNumber], [expiryDate], dan
[cvv]) dienkapsulasi dengan modifier `private`.
 * - Biaya transaksi sebesar 2% dari nominal belanja.
 *
 * @property cardNumber Nomor kartu kredit (minimal 16 karakter)
 * @property expiryDate Tanggal kedaluwarsa kartu (misal: "12/28")
 * @property cvv Tiga digit kode keamanan di belakang kartu
 */
class CreditCardPayment(
    private val cardNumber: String,
    private val expiryDate: String,
    private val cvv: String
) : PaymentMethod {

    override val name: String = "Kartu Kredit"

    /**
     * Menghitung biaya administrasi transaksi kartu kredit sebesar
2% dari jumlah transaksi.
     *
     * @param amount Nominal transaksi
     * @return Biaya transaksi (2% dari amount)
     */
    override fun getFee(amount: Double): Double {
        return amount * 0.02
    }
```

```
    /**
     * Memproses verifikasi kartu kredit:
     * - Nomor kartu harus minimal 16 digit.
     * - CVV harus tepat 3 digit.
     *
     * @param amount Nominal transaksi
     * @return [PaymentResult.Success] jika valid, atau
[PaymentResult.Failed] jika tidak valid
     */
    override fun processPayment(amount: Double): PaymentResult {
        if (cardNumber.length < 16) {
            return PaymentResult.Failed("Nomor kartu tidak valid",
401)
        }
        if (cvv.length != 3) {
            return PaymentResult.Failed("CVV tidak valid", 402)
        }
        return PaymentResult.Success("CC-
${System.currentTimeMillis()}")
    }
}
```

7.10. File `src/payment/QRISPayment.kt`

```
package payment

/**
 * Implementasi metode pembayaran menggunakan QRIS (Quick Response
Code Indonesian Standard).
 *
 * Menerapkan prinsip **Abstraksi** dan **Enkapsulasi**:
 * - Mengimplementasikan kontrak [PaymentMethod].
 * - Properti [qrCode] dan [merchantId] dienkapsulasi dengan
modifier `private`.
 * - Biaya transaksi sebesar 0.5% dari nominal belanja.
 *
 * @property qrCode Kode representasi string payload QRIS (minimal
10 karakter)
 * @property merchantId ID unik merchant e-commerce
 */
class QRISPayment(
    private val qrCode: String,
    private val merchantId: String
) : PaymentMethod {

    override val name: String = "QRIS"

    /**
     * Menghitung biaya administrasi transaksi QRIS sebesar 0.5%
dari nominal transaksi.
     *
     * @param amount Nominal transaksi
     * @return Biaya transaksi (0.5% dari amount)
     */
    override fun getFee(amount: Double): Double {
        return amount * 0.005
    }
```

```
    /**
     * Memproses pembayaran menggunakan QRIS:
     * - Panjang kode QR harus minimal 10 karakter.
     *
     * @param amount Nominal transaksi
     * @return [PaymentResult.Success] jika kode QR valid, atau
[PaymentResult.Failed] jika tidak valid
     */
    override fun processPayment(amount: Double): PaymentResult {
        if (qrCode.length < 10) {
            return PaymentResult.Failed("Kode QR tidak valid", 403)
        }
        return PaymentResult.Success("QR-
${System.currentTimeMillis()}")
    }
}
```

7.11. File `src/payment/BankTransferPayment.kt`

```
package payment

/**
 * Implementasi metode pembayaran melalui Transfer Antar Bank.
 *
 * Menerapkan prinsip **Abstraksi** dan **Enkapsulasi**:
 * - Mengimplementasikan interface [PaymentMethod].
 * - Properti [bankName] dan [accountNumber] dienkapsulasi dengan
modifier `private`.
 * - Biaya transaksi sebesar 1% dari total belanja dengan nominal
minimum sebesar Rp 5.000.
 *
 * @property bankName Nama bank tujuan transfer (contoh: "BCA",
"Mandiri", "BNI")
 * @property accountNumber Nomor rekening bank (minimal 8 karakter)
 */
class BankTransferPayment(
    private val bankName: String,
    private val accountNumber: String
) : PaymentMethod {

    override val name: String = "Transfer Bank"

    /**
     * Menghitung biaya transfer bank yaitu 1% dari nominal belanja,
     * dengan nilai minimum Rp 5.000.
     *
     * @param amount Nominal transaksi
     * @return Biaya transaksi dalam Rupiah
     */
    override fun getFee(amount: Double): Double {
        val fee = amount * 0.01
        return maxOf(fee, 5000.0)
    }

    /**
     * Memproses transfer bank:
     * - Nomor rekening harus minimal 8 digit.
     *
     * @param amount Nominal transaksi
```

```
     * @return [PaymentResult.Success] jika nomor rekening valid,
atau [PaymentResult.Failed] jika tidak valid
     */
    override fun processPayment(amount: Double): PaymentResult {
        if (accountNumber.length < 8) {
            return PaymentResult.Failed("Nomor rekening tidak
valid", 404)
        }
        return PaymentResult.Success("BT-
${System.currentTimeMillis()}")
    }
}
```

7.12. File `src/user/User.kt`

```
package user

import cart.ShoppingCart
import order.Order
import payment.PaymentMethod
import payment.PaymentResult

/**
 * Kelas yang merepresentasikan pengguna (customer) di dalam sistem
e-commerce.
 *
 * Menerapkan prinsip **Enkapsulasi**:
 * - Properti [password] bersifat `private` dan tidak memiliki
getter publik, hanya dapat diverifikasi lewat [authenticate].
 * - Koleksi [orders] dan objek [cart] dienkapsulasi untuk
mengontrol integritas transaksi.
 * - Informasi total pesanan dan total pengeluaran disediakan
melalui properti terhitung [orderCount] dan [totalSpent].
 *
 * @property username Nama unik pengguna untuk login dan
identifikasi
 * @property email Alamat email pengguna
 * @property password Kata sandi akun (dienkapsulasi secara private)
 */
class User(
    val username: String,
    val email: String,
    private val password: String
) {
    // ============================================================
    // PROPERTI
    // ============================================================

    /**
     * Daftar riwayat seluruh pesanan yang pernah dilakukan oleh
pengguna.
     */
    private val orders = mutableListOf<Order>()

    /**
     * Objek keranjang belanja milik pengguna ini.
     */
    private val cart = ShoppingCart(username)
```

```
    /**
     * Jumlah transaksi pesanan yang telah berhasil dibuat.
     */
    val orderCount: Int
        get() = orders.size

    /**
     * Akumulasi nominal uang yang telah dibelanjakan pada seluruh
pesanan.
     */
    val totalSpent: Double
        get() = orders.sumOf { it.totalPrice }

    // ============================================================
    // METODE
    // ============================================================

    /**
     * Memverifikasi keabsahan kata sandi pengguna tanpa membocorkan
kata sandi asli (enkapsulasi).
     *
     * @param inputPassword Kata sandi yang dimasukkan pengguna
     * @return `true` jika kata sandi cocok, `false` jika salah
     */
    fun authenticate(inputPassword: String): Boolean {
        return password == inputPassword
    }

    /**
     * Mengakses objek keranjang belanja milik pengguna.
     *
     * @return Objek [ShoppingCart] milik user
     */
    fun getCart(): ShoppingCart = cart

    /**
     * Melakukan proses checkout belanja dari keranjang menggunakan
metode pembayaran yang dipilih.
     * Menerapkan prinsip Polimorfisme pada parameter
[paymentMethod].
     *
     * @param paymentMethod Objek pembayaran polimorfik yang
mengimplementasikan [PaymentMethod]
     * @return Objek [Order] jika pembayaran berhasil, atau `null`
jika pembayaran gagal / keranjang kosong
     */
    fun checkout(paymentMethod: PaymentMethod): Order? {
        if (cart.isEmpty()) {
            println("❌ Keranjang kosong")
            return null
        }

        val amount = cart.getTotalPrice()
        println("💳 Memproses pembayaran dengan
${paymentMethod.name}...")

        val result = paymentMethod.processPayment(amount)
        return when (result) {
            is PaymentResult.Success -> {
```

```
                println("✅ Pembayaran berhasil! ID:
${result.transactionId}")
                val order = createOrder()
                orders.add(order)
                println("✅ Order ${order.id} berhasil dibuat")
                order
            }
            is PaymentResult.Failed -> {
                println("❌ Pembayaran gagal: ${result.reason}
(Error Code: ${result.errorCode})")
                null
            }
            PaymentResult.Pending -> {
                println("⏳ Pembayaran pending...")
                null
            }
        }
    }

    /**
     * Helper privat untuk membuat objek [Order] baru dari item
keranjang belanja saat ini
     * dan mengosongkan keranjang belanja setelahnya.
     *
     * @return Objek [Order] yang baru dibuat
     */
    private fun createOrder(): Order {
        val orderId = "ORD-${System.currentTimeMillis()}"
        val items = cart.getItems()
        val order = Order(orderId, username, items)
        // Kosongkan keranjang setelah order dibuat
        cart.clear()
        return order
    }

    /**
     * Mengambil daftar salinan riwayat pesanan (read-only list).
     *
     * @return List berisi objek [Order]
     */
    fun getOrders(): List<Order> = orders.toList()

    /**
     * Menampilkan daftar seluruh riwayat pesanan pengguna ke konsol
beserta total belanjaan.
     */
    fun displayOrders() {
        println("=".repeat(50))
        println("📋 RIWAYAT ORDER - $username")
        println("=".repeat(50))

        if (orders.isEmpty()) {
            println("   Belum ada order")
        } else {
            orders.forEachIndexed { index, order ->
                println("${index + 1}. Order ${order.id} -
${order.status.display()}")
                println("   Total: Rp
${formatRupiah(order.totalPrice)}")
            }
```

```
        }
        println("Total Belanja: Rp ${formatRupiah(totalSpent)}")
        println("=".repeat(50))
    }

    /**
     * Helper privat untuk memformat angka nominal menjadi format
Rupiah dengan pemisah titik ribuan.
     */
    private fun formatRupiah(nominal: Double): String {
        val str = nominal.toLong().toString()
        val builder = StringBuilder()
        var count = 0
        for (i in str.length - 1 downTo 0) {
            builder.insert(0, str[i])
            count++
            if (count % 3 == 0 && i > 0) {
                builder.insert(0, ".")
            }
        }
        return builder.toString()
    }
}
```

7.13. File `src/system/ECommerceSystem.kt`

```
package system

import order.Order
import product.Product
import user.User

/**
 * Pengendali utama (Main Controller) untuk sistem manajemen e-
commerce.
 *
 * Menerapkan prinsip **Enkapsulasi**:
 * - Mengelola koleksi produk ([products]), pengguna ([users]), dan
pesanan ([orders]) secara privat.
 * - Menyediakan API publik terstruktur untuk manajemen produk,
autentikasi/registrasi user,
 *   dan pelaporan penjualan sistem.
 *
 * @property name Nama platform toko online (contoh: "Toko Online
Kampus")
 */
class ECommerceSystem(val name: String = "Toko Online") {

    // ============================================================
    // PROPERTI
    // ============================================================

    /**
     * Koleksi produk yang terdaftar di dalam katalog toko.
     */
    private val products = mutableListOf<Product>()

    /**
     * Koleksi pengguna yang terdaftar di dalam sistem.
```

```
     */
    private val users = mutableListOf<User>()

    /**
     * Koleksi seluruh pesanan yang telah diselesaikan di dalam
sistem toko.
     */
    private val orders = mutableListOf<Order>()

    /**
     * Jumlah keseluruhan pesanan yang tercatat dalam sistem.
     */
    val orderCount: Int
        get() = orders.size


    // ============================================================
    // MANAJEMEN PRODUK
    // ============================================================

    /**
     * Menambahkan produk baru ke katalog toko online.
     *
     * @param product Objek [Product] yang akan ditambahkan
     */
    fun addProduct(product: Product) {
        products.add(product)
        println("✅ Produk ${product.name} ditambahkan")
    }

    /**
     * Mencari produk berdasarkan kata kunci yang dicocokkan dengan
nama atau ID produk (case-insensitive).
     *
     * @param keyword Kata kunci pencarian
     * @return List produk yang sesuai dengan kriteria pencarian
     */
    fun searchProduct(keyword: String): List<Product> {
        return products.filter {
            it.name.lowercase().contains(keyword.lowercase()) ||
            it.id.lowercase().contains(keyword.lowercase())
        }
    }

    /**
     * Menemukan produk berdasarkan ID uniknya.
     *
     * @param id ID produk yang dicari
     * @return Objek [Product] jika ditemukan, atau `null` jika
tidak ditemukan
     */
    fun findProductById(id: String): Product? {
        return products.find { it.id.equals(id, ignoreCase = true) }
    }

    /**
     * Mengambil seluruh daftar produk terdaftar sebagai read-only
list.
     *
     * @return List seluruh objek [Product]
     */
```

```
    fun getAllProducts(): List<Product> = products.toList()

    /**
     * Menampilkan seluruh katalog produk ke konsol dengan memanggil
[Product.displayInfo].
     */
    fun displayAllProducts() {
        println("=".repeat(55))
        println("📦 SEMUA PRODUK")
        println("=".repeat(55))
        println("Total: ${products.size} produk")
        println("-".repeat(55))

        products.forEach { product ->
            product.displayInfo()
        }
    }

    // ============================================================
    // MANAJEMEN USER
    // ============================================================

    /**
     * Mendaftarkan pengguna baru ke dalam sistem setelah
memvalidasi keunikan username.
     *
     * @param username Nama pengguna baru
     * @param email Alamat email
     * @param password Kata sandi akun pengguna
     * @return `true` jika registrasi berhasil, `false` jika
username sudah terpakai
     */
    fun registerUser(username: String, email: String, password:
String): Boolean {
        if (users.any { it.username.equals(username, ignoreCase =
true) }) {
            println("❌ Username $username sudah digunakan")
            return false
        }
        val user = User(username, email, password)
        users.add(user)
        println("✅ User $username berhasil didaftarkan")
        return true
    }

    /**
     * Menemukan akun pengguna berdasarkan nama pengguna.
     *
     * @param username Nama pengguna yang dicari
     * @return Objek [User] jika ditemukan, atau `null` jika tidak
terdaftar
     */
    fun findUser(username: String): User? {
        return users.find { it.username.equals(username, ignoreCase
= true) }
    }

    // ============================================================
    // MANAJEMEN ORDER & LAPORAN
    // ============================================================
```

```
    /**
     * Menambahkan pesanan baru ke dalam rekapitulasi transaksi
toko.
     *
     * @param order Objek [Order] yang telah berhasil dibuat
     */
    fun addOrder(order: Order) {
        orders.add(order)
    }

    /**
     * Menghitung total keseluruhan pendapatan (revenue) dari semua
order yang tercatat.
     *
     * @return Akumulasi pendapatan dalam satuan Rupiah
     */
    fun getTotalRevenue(): Double {
        return orders.sumOf { it.totalPrice }
    }

    /**
     * Mengambil jumlah pesanan yang ada dalam sistem.
     *
     * @return Total pesanan
     */
    @JvmName("fetchOrderCount")
    fun getOrderCount(): Int = orders.size


    /**
     * Menampilkan laporan penjualan toko online ke konsol,
     * merangkum total pesanan, total omset/revenue, dan daftar
riwayat tiap transaksi.
     */
    fun displaySalesReport() {
        println("=".repeat(55))
        println("📊 LAPORAN PENJUALAN")
        println("=".repeat(55))
        println("Total Order   : $orderCount")
        println("Total Revenue : Rp
${formatRupiah(getTotalRevenue())}")
        println("-".repeat(55))

        if (orders.isEmpty()) {
            println("   Belum ada penjualan")
        } else {
            orders.forEachIndexed { index, order ->
                println("${index + 1}. ${order.id} -
${order.customerName}")
                println("   Status: ${order.status.display()}")
                println("   Total: Rp
${formatRupiah(order.totalPrice)}")
            }
        }
        println("=".repeat(55))
    }

    /**
     * Helper privat untuk memformat angka nominal menjadi format
Rupiah dengan pemisah ribuan.
```

```
     */
    private fun formatRupiah(nominal: Double): String {
        val str = nominal.toLong().toString()
        val builder = StringBuilder()
        var count = 0
        for (i in str.length - 1 downTo 0) {
            builder.insert(0, str[i])
            count++
            if (count % 3 == 0 && i > 0) {
                builder.insert(0, ".")
            }
        }
        return builder.toString()
    }
}
```

7.14. File `src/Main.kt`

```
import cart.ShoppingCart
import order.Order
import order.OrderStatus
import payment.BankTransferPayment
import payment.CreditCardPayment
import payment.PaymentMethod
import payment.PaymentResult
import payment.QRISPayment
import product.ClothingProduct
import product.ElectronicProduct
import product.FoodProduct
import product.Product
import system.ECommerceSystem
import user.User

/**
 * File program utama untuk mendemonstrasikan implementasi seluruh
spesifikasi
 * Sistem Manajemen E-Commerce berbasis OOP pada Tugas Kelompok PBO.
 *
 * Mendemonstrasikan 14 skenario fitur & pilar OOP:
 * 1. Inisialisasi Sistem
 * 2. Penambahan Produk (Elektronik, Pakaian, Makanan)
 * 3. Registrasi Pengguna
 * 4. Katalog Produk
 * 5. Pengelolaan Keranjang Belanja User 1
 * 6. Tampilan & Kalkulasi Diskon Keranjang User 1
 * 7. Proses Checkout & Pembayaran Kartu Kredit User 1
 * 8. Rincian & Transisi Status Pesanan User 1
 * 9. Belanja & Pembayaran QRIS User 2
 * 10. Rekapitulasi Laporan Penjualan Toko
 * 11. Demonstrasi Polimorfisme (Polymorphic references & dynamic
dispatch)
 * 12. Demonstrasi Sealed Class (Exhaustive when & status guard)
 * 13. Demonstrasi Enkapsulasi (Data hiding & private accessors)
 * 14. Demonstrasi Smart Casting (`is` check & safe cast `as?`)
 */
fun main() {
    println("=".repeat(55))
    println("🛍️  SELAMAT DATANG DI TOKO ONLINE KAMPUS")
    println("=".repeat(55))
    println()
```

```
// ============================================================
// 1. Inisialisasi Sistem
// ============================================================
println(">>> 1. INISIALISASI SISTEM E-COMMERCE <<<")
val system = ECommerceSystem("Toko Online Kampus")
println("Sistem '${system.name}' berhasil diinisialisasi.\n")

// ============================================================
// 2. Menambahkan Produk (Minimal 6 produk: 2 per kategori)
// ============================================================
println("--- MENAMBAHKAN PRODUK ---")
// Kategori Elektronik (1 premium, 1 non-premium)
val laptopAsus = ElectronicProduct(
    id = "E001",
    name = "Laptop Gaming ASUS ROG",
    price = 15000000.0,
    stock = 10,
    brand = "ASUS",
    warrantyMonths = 36,
    isPremium = true
)
val smartphoneXiaomi = ElectronicProduct(
    id = "E002",
    name = "Smartphone Xiaomi Redmi",
    price = 2500000.0,
    stock = 15,
    brand = "Xiaomi",
    warrantyMonths = 12,
    isPremium = false
)

// Kategori Pakaian (1 seasonal, 1 non-seasonal)
val jaketWinter = ClothingProduct(
    id = "C001",
    name = "Jaket Musim Dingin Parka",
    price = 400000.0,
    stock = 20,
    size = "XL",
    material = "Wol & Dacron",
    isSeasonal = true
)
val kaosPolos = ClothingProduct(
    id = "C002",
    name = "Kaos Polos Cotton Combed 30s",
    price = 100000.0,
    stock = 50,
    size = "L",
    material = "Katun Combed",
    isSeasonal = false
)

// Kategori Makanan (1 organik, 1 non-organik)
val berasOrganik = FoodProduct(
    id = "F001",
    name = "Beras Organik Raja Pandan",
    price = 85000.0,
    stock = 30,
    expiryDate = "2026-12-31",
    weight = 5000.0,
    isOrganic = true
```

```
    )
    val mieInstan = FoodProduct(
        id = "F002",
        name = "Mie Instan Goreng Spesial",
        price = 3500.0,
        stock = 100,
        expiryDate = "2026-06-30",
        weight = 85.0,
        isOrganic = false
    )

    // Daftarkan semua produk ke sistem
    system.addProduct(laptopAsus)
    system.addProduct(smartphoneXiaomi)
    system.addProduct(jaketWinter)
    system.addProduct(kaosPolos)
    system.addProduct(berasOrganik)
    system.addProduct(mieInstan)
    println()

    // ============================================================
    // 3. Registrasi User (Minimal 2 user)
    // ============================================================
    println("--- REGISTRASI USER ---")
    system.registerUser("budi", "budi@kampus.ac.id",
"rahasiaBudi123")
    system.registerUser("siti", "siti@kampus.ac.id", "sitiAman456")
    // Uji proteksi duplikasi username
    system.registerUser("budi", "budi.baru@kampus.ac.id",
"passwordLain")
    println()

    // Dapatkan instance user untuk simulasi transaksi
    val userBudi = system.findUser("budi") ?: error("User budi tidak
ditemukan")
    val userSiti = system.findUser("siti") ?: error("User siti tidak
ditemukan")

    // ============================================================
    // 4. Tampilkan Semua Produk
    // ============================================================
    println("--- KATALOG SEMUA PRODUK ---")
    system.displayAllProducts()
    println()

    // ============================================================
    // 5. User 1 (Budi): Menambahkan Produk ke Keranjang
    // ============================================================
    println("--- BUDI MENAMBAH BARANG KE KERANJANG ---")
    val cartBudi = userBudi.getCart()
    cartBudi.addItem(laptopAsus, 1)     // 1 unit Laptop Gaming
    cartBudi.addItem(jaketWinter, 2)    // 2 unit Jaket Musim Dingin
    cartBudi.addItem(mieInstan, 5)      // 5 bungkus Mie Instan
    // Uji validasi penambahan stok melebihi batas
    println("Mencoba memesan barang melebihi stok yang tersedia:")
    cartBudi.addItem(laptopAsus, 999)
    println()

    // ============================================================
    // 6. User 1 (Budi): Tampilkan Keranjang
    // ============================================================
```

```
    println("--- TAMPILAN KERANJANG BELANJA BUDI ---")
    cartBudi.displayCart()
    println()

    // ============================================================
    // 7. User 1 (Budi): Checkout dengan Metode Pembayaran
    // ============================================================
    println("--- CHECKOUT BUDI (KARTU KREDIT) ---")
    val ccPayment = CreditCardPayment(
        cardNumber = "4111222233334444",
        expiryDate = "12/28",
        cvv = "888"
    )
    val feeBudi = ccPayment.getFee(cartBudi.getTotalPrice())
    println("Metode Pembayaran: ${ccPayment.name}")
    println("Biaya Layanan (${ccPayment.name}): Rp
${formatRupiah(feeBudi)}")

    val orderBudi = userBudi.checkout(ccPayment)
    if (orderBudi != null) {
        system.addOrder(orderBudi)
    }
    println()

    // ============================================================
    // 8. Tampilkan Order User 1 & Demonstrasi Status Order
    // ============================================================
    println("--- STATUS DAN DETAIL ORDER BUDI ---")
    userBudi.displayOrders()
    println()
    if (orderBudi != null) {
        orderBudi.displayOrder()
        println("\n>>> Demonstrasi Pembaruan Status Pesanan <<<")
        orderBudi.updateStatus(OrderStatus.Paid)
        orderBudi.updateStatus(OrderStatus.Shipped)
        orderBudi.updateStatus(OrderStatus.Delivered)
        println("Mencoba mengubah status pesanan yang sudah
berstatus final (Delivered):")
        orderBudi.updateStatus(OrderStatus.Cancelled("Pelanggan
ingin refund"))
    }
    println()

    // ============================================================
    // 9. User 2 (Siti): Belanja & Checkout dengan QRIS
    // ============================================================
    println("--- SITI MENAMBAH BARANG KE KERANJANG ---")
    val cartSiti = userSiti.getCart()
    cartSiti.addItem(berasOrganik, 2)       // 2 karung Beras
Organik
    cartSiti.addItem(kaosPolos, 3)          // 3 pcs Kaos Polos
    cartSiti.displayCart()
    println()

    println("--- CHECKOUT SITI (QRIS) ---")
    val qrisPayment = QRISPayment(
        qrCode =
"00020101021126580014ID.GO.QRIS.WWW.KAMPUS.NMID012345",
        merchantId = "MERCHANT_KAMPUS_01"
    )
    val feeSiti = qrisPayment.getFee(cartSiti.getTotalPrice())
```

```
    println("Metode Pembayaran: ${qrisPayment.name}")
    println("Biaya Layanan (${qrisPayment.name}): Rp
${formatRupiah(feeSiti)}")

    val orderSiti = userSiti.checkout(qrisPayment)
    if (orderSiti != null) {
        system.addOrder(orderSiti)
        orderSiti.updateStatus(OrderStatus.Paid)
        orderSiti.updateStatus(OrderStatus.Shipped)
    }
    println()
    userSiti.displayOrders()
    println()

    // ============================================================
    // 10. Tampilkan Laporan Penjualan Sistem
    // ============================================================
    println("--- LAPORAN PENJUALAN TOKO KAMPUS ---")
    system.displaySalesReport()
    println()

    // ============================================================
    // 11. Demonstrasi Polimorfisme (Polymorphism)
    // ============================================================
    println("=".repeat(55))
    println("🧠 11. DEMONSTRASI PILAR OOP: POLIMORFISME")
    println("=".repeat(55))
    println("Menampung berbagai jenis subclass ke dalam
List<Product> (Polymorphic Reference):")
    val polymorphicProductList: List<Product> = listOf(
        laptopAsus,
        jaketWinter,
        berasOrganik
    )

    polymorphicProductList.forEach { product ->
        // Pemanggilan method calculateDiscount() yang berjalan
dinamis sesuai tipe asli objek (Dynamic Dispatch)
        val diskon = product.calculateDiscount()
        val hargaAkhir = product.getDiscountedPrice()
        println("Produk: ${product.name}")
        println("  -> Kategori  : ${product.getCategory()}")
        println("  -> Harga Asli: ${product.formattedPrice}")
        println("  -> Diskon    : Rp ${formatRupiah(diskon)}")
        println("  -> Tagihan   : Rp ${formatRupiah(hargaAkhir)}")
    }
    println()

    // Polimorfisme pada interface PaymentMethod
    println("Polimorfisme pada gerbang pembayaran (PaymentMethod):")
    val paymentList: List<PaymentMethod> = listOf(
        CreditCardPayment("1111222233334444", "10/27", "777"),
        QRISPayment("0123456789QRISPAY", "M_001"),
        BankTransferPayment("BCA", "1234567890")
    )
    val testAmount = 500000.0
    paymentList.forEach { method ->
        println("- ${method.name}: Fee untuk transaksi Rp
${formatRupiah(testAmount)} adalah Rp
${formatRupiah(method.getFee(testAmount))}")
    }
```

```
    println()

    // ============================================================
    // 12. Demonstrasi Sealed Class
    // ============================================================
    println("=".repeat(55))
    println("🔒 12. DEMONSTRASI SEALED CLASS (OrderStatus &
PaymentResult)")
    println("=".repeat(55))
    val sampleStatuses: List<OrderStatus> = listOf(
        OrderStatus.Pending,
        OrderStatus.Paid,
        OrderStatus.Shipped,
        OrderStatus.Delivered,
        OrderStatus.Cancelled("Stok habis di gudang pusat")
    )

    println("Evaluasi exhaustive 'when' pada sealed class
OrderStatus:")
    sampleStatuses.forEach { status ->
        // Exhaustive when tanpa perlu else branch karena semua
cabang sealed class tercover
        val statusMessage = when (status) {
            is OrderStatus.Pending -> "Tahap 1: Pembeli belum
mentransfer dana."
            is OrderStatus.Paid -> "Tahap 2: Dana sudah masuk,
pesanan siap dipacking."
            is OrderStatus.Shipped -> "Tahap 3: Paket sedang dalam
perjalanan kurir."
            is OrderStatus.Delivered -> "Tahap 4: Transaksi selesai,
barang diterima."
            is OrderStatus.Cancelled -> "Tahap Pembatalan: Transaksi
gugur (Alasan: ${status.reason})."
        }
        println("Status [${status.display()}] -> $statusMessage
(Final? ${if (status.isFinal()) '✅' else '❌'})")
    }
    println()

    // ============================================================
    // 13. Demonstrasi Enkapsulasi (Encapsulation)
    // ============================================================
    println("=".repeat(55))
    println("🛡️  13. DEMONSTRASI PILAR OOP: ENKAPSULASI")
    println("=".repeat(55))
    println("1. Perlindungan Password Pengguna:")
    println("   - Properti 'password' pada kelas User di-set
'private'.")
    println("   - Memverifikasi login dengan password salah
('salah123'): ${userBudi.authenticate("salah123")}")
    println("   - Memverifikasi login dengan password benar
('rahasiaBudi123'): ${userBudi.authenticate("rahasiaBudi123")}")
    println()

    println("2. Enkapsulasi Harga (Product.price):")
    println("   - 'price' dienkapsulasi dengan access modifier
protected sehingga tidak dapat dimodifikasi bebas dari luar kelas.")
    println("   - Konsumen luar hanya dapat membaca harga terformat
melalui 'formattedPrice': ${laptopAsus.formattedPrice}")
    println()
```

```
    println("3. Enkapsulasi Total Items pada Keranjang Belanja:")
    println("   - Properti 'totalItems' menggunakan 'private set'.")
    println("   - Nilai total items saat ini:
${userBudi.getCart().totalItems} item.")
    println("   - Pihak luar tidak dapat mengubah totalItems secara
langsung tanpa melalui fungsi addItem/removeItem.")
    println()

    // ============================================================
    // 14. Demonstrasi Smart Casting & Safe Casting
    // ============================================================
    println("=".repeat(55))
    println("⚡ 14. DEMONSTRASI SMART CASTING & SAFE CASTING")
    println("=".repeat(55))

    val mixedCatalog: List<Any> = listOf(
        laptopAsus,
        jaketWinter,
        berasOrganik,
        "Produk Promosi Tambahan"
    )

    println("A. Smart Casting Otomatis menggunakan pemeriksaan tipe
'is':")
    for (item in mixedCatalog) {
        when (item) {
            is ElectronicProduct -> {
                // Di dalam blok ini, 'item' otomatis di-smart-cast
menjadi ElectronicProduct
                println("✨ [Elektronik] Brand: ${item.brand},
Garansi: ${item.warrantyMonths} bulan, Premium: ${item.isPremium}")
            }
            is ClothingProduct -> {
                // Di dalam blok ini, 'item' otomatis di-smart-cast
menjadi ClothingProduct
                println("✨ [Pakaian] Ukuran: ${item.size}, Bahan:
${item.material}, Seasonal: ${item.isSeasonal}")
            }
            is FoodProduct -> {
                // Di dalam blok ini, 'item' otomatis di-smart-cast
menjadi FoodProduct
                println("✨ [Makanan] Berat: ${item.weight}g,
Expired: ${item.expiryDate}, Organik: ${item.isOrganic}")
            }
            else -> {
                println("ℹ️  [Objek Lain]: '$item' bukan turunan
dari kelas Product.")
            }
        }
    }
    println()

    println("B. Safe Casting yang aman menggunakan operator 'as?':")
    val testObj1: Any = laptopAsus
    val testObj2: Any = "Bukan produk elektronik"

    val castResult1: ElectronicProduct? = testObj1 as?
ElectronicProduct
```

```
    val castResult2: ElectronicProduct? = testObj2 as?
ElectronicProduct

    println("Hasil safe cast (testObj1 as? ElectronicProduct):
${castResult1?.name ?: "Gagal cast"}")
    println("Hasil safe cast (testObj2 as? ElectronicProduct):
${castResult2?.name ?: "Gagal cast (mengembalikan null tanpa
Crash/Exception)"}")

    println()
    println("=".repeat(55))
    println("🎉 SELURUH DEMONSTRASI FITUR OOP BERHASIL DIJALANKAN!")
    println("=".repeat(55))
}

/**
 * Helper fungsi mandiri di file Main untuk memformat mata uang
Rupiah.
 */
private fun formatRupiah(nominal: Double): String {
    val str = nominal.toLong().toString()
    val builder = StringBuilder()
    var count = 0
    for (i in str.length - 1 downTo 0) {
        builder.insert(0, str[i])
        count++
        if (count % 3 == 0 && i > 0) {
            builder.insert(0, ".")
        }
    }
    return builder.toString()
}
```